How the Application Is Connected

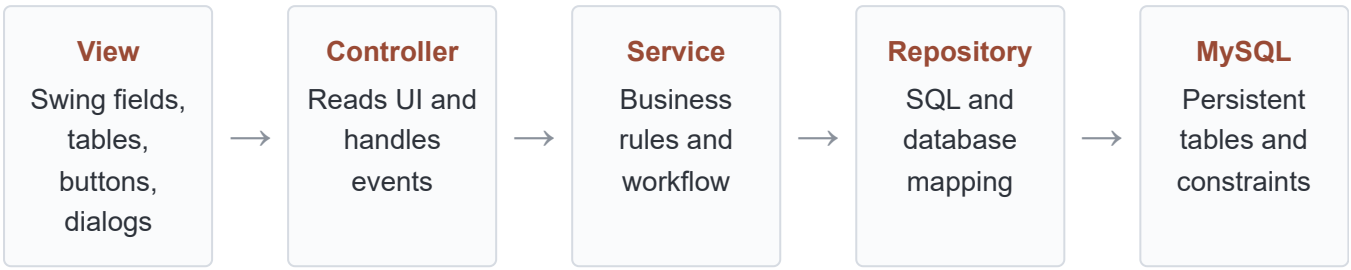
MVC + Service + Repository Architecture

A practical guide based on the current Pastry and Delicious Aliments Java code.

This guide explains:

- what Models, Views, Controllers, Services and Repositories do;
- how `AdminMain` connects the layers;
- how a button click reaches MySQL and returns to the screen;
- why repository interfaces make unit testing easier;
- where the current project follows the architecture and where it can improve.

1. The architecture in one picture



The response travels back in the opposite direction:

```
MySQL result
  -> Repository returns a Model, List, or ErrorCodes
  -> Service interprets the result
  -> Controller chooses the UI response
  -> View displays data or a message
```

The Model travels between layers

`UserModel`, `StoreManagerModel`, and `CustomerModel` carry data. They do not perform SQL and do not display Swing dialogs.

Layer	Knows about	Should not do
View	Swing components and controller-facing listeners/getters	SQL, repository calls, business decisions
Controller	View, service, UI event flow	Write SQL or contain large business rules
Service	Models, business rules, repository interfaces	Read text fields or show dialogs
Repository	Models, JDBC, SQL, table mapping	Know about JFrame/controller behavior
Model	Application data	Open database connections or windows

2. AdminMain is the composition root

`AdminMain.AMain(...)` is the place where concrete objects are created and connected. This is sometimes called the *composition root*.

Step 1: construct concrete repositories

```
IUserRepository userRepository =
        new DBUserRepository(dbConnection);

IStoreManagerRepository storeRepository =
        new DBStoreManagerRepository(dbConnection);

ICustomerRepository customerRepository =
        new DBCustomerRepository(dbConnection);
```

The variables use interface types, while the constructed objects are JDBC implementations. This separates what the application needs from how MySQL provides it.

Step 2: inject repositories into services

```
CreateRestaurant createService =
        new CreateRestaurant(storeRepository, userRepository);

ShowRestaurantService showService =
        new ShowRestaurantService(storeRepository, userRepository);
```

Step 3: create views

```
AdminView adminView = new AdminView();
CreateRestaurantView createView = new CreateRestaurantView();
ShowRestaurantsView restaurantsView = new ShowRestaurantsView();
ShowCustomersView customersView = new ShowCustomersView();
```

Step 4: inject service and view into each controller

```
new CreateRestaurantController(createView, createService);
new ShowRestaurantsController(showService, restaurantsView);
```

Dependency injection: constructors receive dependencies rather than constructing them internally. A controller receives its view and service. A service receives repository interfaces. This makes dependencies visible and replaceable.

3. Repository layer

A repository provides operations for one kind of stored data. It hides JDBC details from the service.

Interface: the contract

```
public interface IStoreManagerRepository {
    StoreManagerModel findById(int id);
    List<StoreManagerModel> findAll();
    ErrorCodes save(StoreManagerModel restaurant);
    ErrorCodes update(StoreManagerModel restaurant);
    ErrorCodes deleteById(int id);
    StoreManagerModel findByOwnerId(int ownerId);
    StoreManagerModel findByNCAP(
        String name, String city,
        String addressLine1, String postalCode);
}
```

The service depends on this interface, not on `DBStoreManagerRepository`. The interface says what is available without exposing SQL.

Implementation: JDBC and SQL

```
public StoreManagerModel findById(int id) {
    String sql = "SELECT " + SELECT_COLUMNS
               + " FROM restaurants WHERE id = ?";

    try (PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setInt(1, id);
        try (ResultSet resultSet = statement.executeQuery()) {
            return resultSet.next()
                    ? mapStoreManager(resultSet)
                    : null;
        }
    } catch (SQLException exception) {
        throw databaseException("find restaurant by ID", exception);
    }
}
```

What this method does

1. Builds a parameterized SQL query.
2. Binds the ID safely through `PreparedStatement`.
3. Executes the query.
4. Maps a database row into `StoreManagerModel`.
5. Returns `null` when the ID does not exist.
6. Closes the statement and result set automatically.

> **Important distinction:** “not found” is a normal result. A closed connection, invalid SQL, or foreign-key

violation is an exceptional database failure.

4. Service layer

A service expresses an application use case. It can coordinate multiple repositories and enforce rules that do not belong in the UI or SQL mapper.

Example: CreateRestaurant uses two repositories

```
public CreateRestaurant(
        IStoreManagerRepository storeManagerRepo,
        IUserRepository userRepo) {
    this.storeManagerRep = storeManagerRepo;
    this.userRep = userRepo;
}
```

The creation workflow spans both the `users` and `restaurants` tables, so one service coordinates both repositories:

1. Force the user type to `restaurant_owner`.
2. Check whether the same restaurant location already exists.
3. Find the user by username.
4. Create a new user or validate the existing user's type.
5. Copy the generated user ID into `restaurant.owner_id`.
6. Save the restaurant.
7. Return an `ErrorCodes` value.

```
StoreManagerModel existing = storeManagerRep.findByNCAP(
        restaurant.getName(),
        restaurant.getCity(),
        restaurant.getAddress_line1(),
        restaurant.getPostal_code());

if (existing != null) {
    return ErrorCodes.ALREADY_EXISTS;
}

UserModel existingUser = userRep.findByUsername(user.getUsername());
// Create or validate user...

restaurant.setOwner_id(existingUser.getUserId());
storeManagerRep.save(restaurant);
return ErrorCodes.SUCCESS;
```

ErrorCodes is a layer-neutral result

The service does not show “Restaurant created successfully.” It returns `ErrorCodes.SUCCESS`. The controller translates that result into user-facing text.

5. Controller layer

A controller connects UI events to services. It reads view input, constructs models, calls the service, and selects the appropriate view response.

Listeners are attached in the constructor

```
public CreateRestaurantController(
        CreateRestaurantView view,
        CreateRestaurant service) {
    this.view = view;
    this.service = service;

    view.addCreateListener(event -> createRestaurant());
}
```

Form values become models

```
UserModel user = new UserModel();
user.setUsername(view.getUsernameInput());
user.setUserEmail(view.getEmailInput());
user.setUserPassword(view.getPasswordInput());
user.setUserPhone(view.getPhoneInput());

StoreManagerModel restaurant = new StoreManagerModel();
restaurant.setName(view.getRestaurantNameInput());
restaurant.setAddress_line1(view.getAddressInput());
restaurant.setCity(view.getCityInput());
restaurant.setPostal_code(view.getPostalCodeInput());
```

The service result becomes a UI message

```
ErrorCodes result = service.createRestaurant(user, restaurant);

switch (result) {
    case SUCCESS ->
        view.showMessage("Restaurant created successfully.");
    case ALREADY_EXISTS ->
        view.showMessage("This restaurant already exists.");
    case BAD_TYPE ->
        view.showMessage("The user type is invalid.");
    default ->
        view.showMessage("The operation failed.");
}
```

The controller knows the text appropriate for the current screen. The service remains reusable by another controller or by unit tests.

6. View layer

A view owns Swing components and exposes a small public API to its controller.

Expose events rather than service calls

```
public void addCreateListener(ActionListener listener) {
    createButton.addActionListener(listener);
}
```

Expose input as plain values

```
public String getRestaurantNameInput() {
    return restaurantNameField.getText().trim();
}
```

Accept data from outside

```
public void setRestaurants(List<StoreManagerModel> restaurants) {
    tableModel.setRestaurants(restaurants);
}
```

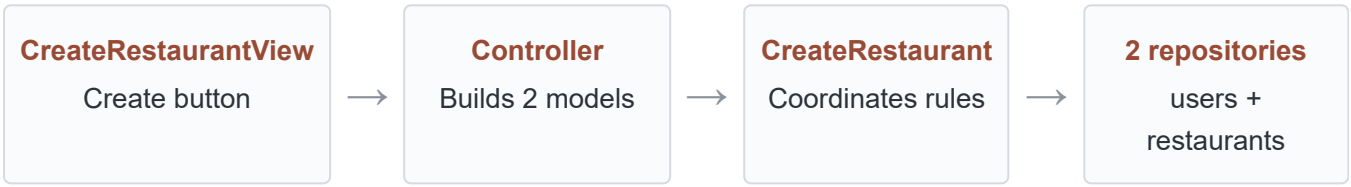
Expose the selected model

```
public StoreManagerModel getSelectedRestaurant() {
    stopCellEditing();
    int row = restaurantTable.getSelectedRow();
    return row < 0 ? null : tableModel.getRestaurantAt(row);
}
```

This pattern gives the controller enough power to coordinate behavior without exposing every private Swing field.

7. End-to-end example: Create Restaurant

```
1. User enters form values in CreateRestaurantView.
2. User presses Create.
3. Swing invokes the controller's registered ActionListener.
4. CreateRestaurantController reads the view getters.
5. Controller creates UserModel and StoreManagerModel.
6. Controller calls CreateRestaurant.createRestaurant(...).
7. Service checks duplicates through IStoreManagerRepository.
8. Service finds/saves the user through IUserRepository.
9. DBUserRepository executes SQL against users.
10. DBStoreManagerRepository executes SQL against restaurants.
11. Repositories return models/ErrorCodes.
12. Service returns ErrorCodes to the controller.
13. Controller calls view.showMessage(...).
```



8. End-to-end example: Show and update restaurants

Loading the table

```
Controller.RefreshRestaurantList()
  -> service.getAllRestaurants()
  -> repository.findAll()
  -> SELECT ... FROM restaurants
  -> List<StoreManagerModel>
  -> view.setRestaurants(list)
```

Saving an edited row

```
public void saveRestaurant() {
    StoreManagerModel restaurant = view.getSelectedRestaurant();
    ErrorCodes result = service.UpdateRestaurant(restaurant);
    // Show result message, then refresh the list.
}
```

The table edits the selected model's fields. The controller retrieves that model and passes it to the service. The repository executes an `UPDATE ... WHERE id = ?`.

Showing details

```
StoreManagerModel restaurant = view.getSelectedRestaurant();
ShowRestaurantsView.showRestaurantDetails(view, restaurant);
```

This does not need the database if the table already holds all required fields. Refreshing before reading the selection would clear the selection.

9. Why repository interfaces help unit tests

A service test should not require MySQL. Mockito creates objects implementing the repository interfaces:

```
@Mock
private IStoreManagerRepository storeManagerRepository;

@Mock
private IUserRepository userRepository;

private CreateRestaurant service;

@BeforeEach
void setUp() {
    service = new CreateRestaurant(
        storeManagerRepository,
        userRepository);
}
```

The test controls repository answers:

```
when(storeManagerRepository.findByNCAP(
        "Pastry House", "Athens",
        "10 Baker Street", "10558"))
    .thenReturn(new StoreManagerModel());

ErrorCodes result = service.createRestaurant(user, restaurant);

assertEquals(ErrorCodes.ALREADY_EXISTS, result);
verify(storeManagerRepository, never()).save(restaurant);
```

The test checks only service decisions. JDBC integration can be tested separately.

10. Current project boundaries and improvements

The project largely follows the intended dependency direction, but two current areas bypass it:

Controller accesses a repository through the service

```
List<UserModel> users = service.getUserRep().findAllUsers();
```

This makes the controller aware of repository behavior. A cleaner design would expose a service method such as `getAllUsers()`, allowing the service to remain the controller's only data dependency.

AdminMain loads customers directly from a repository

```
List<CustomerModel> allCustomers = customerRepository.findAll();
showCustomersView.setCustomers(allCustomers);
```

The comment already marks this as a UI experiment. In the final design, a customer service and controller should load the list.

Business-operation transactions

Creating a restaurant writes both a user and a restaurant. Ideally those writes share one database transaction so that either both succeed or both roll back. This is a future improvement; it does not change the responsibility of each layer.

Naming consistency

Java convention uses lowercase packages and lower camel case method names, for example `com.admin`, `updateRestaurant()`, `deleteRestaurant()`, and `refreshRestaurantList()`.

11. Rules of thumb

If you are writing...	Put it in...
JTextField, JTable, JFrame, JOptionPane	View
Button listener, reading form values, choosing a message	Controller
Duplicate check, user-type rule, multi-step workflow	Service
SELECT, INSERT, UPDATE, DELETE, ResultSet mapping	Repository implementation
Fields describing a user/customer/restaurant	Model
Creating and connecting all objects	Application entry point / composition root

Final dependency direction

```
AdminMain
  -> constructs everything

Controller
  -> View
  -> Service

Service
  -> Repository interfaces
  -> Models

DB Repository
  -> JDBC / MySQL
  -> Models

View
  -> Swing
  -> Models used for display
```

The core idea: each layer should have one reason to change. UI redesign changes the view; business-rule changes affect the service; SQL/schema changes affect repositories.